Data Wrangling 🤠

Types

Lesson 2

How is data stored?

- All data is stored as bits (i.e. 0 or 1)
- **Bits** can be chained to represent decimal numbers
- 000 = 0
  001 = 1
  010 = 2
  011 = 3
  100 = 4

Types

- The string *01000001* is used to represent both the number **65** and the letter **A**
- Programmers must specify how a computer should interpret the underlying bit pattern

Integers: `int`

- Integers are represented as `int`
- Must begin with a non-zero digit
- Has no decimal digits
- Examples: `99`, `100`, `100000`

```
>>> type(99)
<class 'int'>
>>> type(100)
<class 'int'>
>>> type(100000)
<class 'int'>
```

Floating Point Numbers: `float`

- "Decimals" are represented as `float`
- We have finite storage and decimals can be infinite
- Examples: `0.1`, `99.9`, `100.0`

```
>>> type(99.9)
<class 'float'>
>>> type(100.0)
<class 'float'>
>>> 0.1 + 0.2
0.30000000000000004
```

Character Strings: `str`

- Textual data is represented as `str`
- Denoted with opening and closing quotation marks
- Examples: `"cowboy"`, `"99"`, `" "`, `""`

```
>>> type("cowboy")
<class 'str'>
>>> type("99")
<class 'str'>
>>> type(" ")
<class 'str'>
>>> type("")
<class 'str'>
```

Indexing Strings

- Strings are sequences of characters - indices begin at 0
- You can **index** positions in the sequence using square brackets

```
>>> "cowboy"[0]
'c'
>>> "cowboy"[1]
'o'
>>>  "cat"[3]
Traceback (most recent call last):
  File "<python-input-2>", line 1, in <module>
    "cat"[3]
    ~~~~~^^^
IndexError: string index out of range
```

Booleans: `bool`

- Logical data is represented as `bool`
- A bool has 2 possible values
  Examples: `True`, `False`

```
>>> type(True)
<class 'bool'>
>>> type(False)
<class 'bool'>
```